

Une histoire de la réverbération algorithmique de Schroeder à la Freeverb

MAGGUILLI Séverin, POULALION Léo-Polde

Décembre 2023



Table des matières

1	Introduction	3
2	La première réverbération algorithmique	3
2.1	Une réverbe incolore	3
2.2	Augmentation du réalisme	4
3	Les améliorations de Moorer	5
3.1	Les défauts du réverbérateur de Schroeder	6
3.2	Vers une réverb plus réaliste	6
3.3	Un problème inhérent à la conception du réverbérateur	8
4	La Freeverb, un modèle de réverbération plutôt classique	10
4.1	Le combfilter	10
4.1.1	Processing audio	10
4.1.2	Implémentation fonctionnelle	10
4.1.3	Fonction de transfert $H_{cf}(z)$	10
4.2	Le allpass filter	13
4.2.1	Processing audio	13
4.2.2	Implémentation fonctionnelle	14
4.2.3	La fonction de transfert $H_{ap}(z)$	14
4.3	Tuning de la reverb	17
4.4	Architecture globale de la Freeverb	17
5	ReaVerbate, un VST utilisant le processing de le Freeverb	18
5.1	La stereo width ou largeur stéréo	18
5.2	Le damping ou amortissement	20
5.3	Le feedback	22
5.4	Conclusion	23
A	Annexe	24
A.1	le code "tuning.h"	25
A.2	La fonction update	26
A.3	Ressources	27
A.3.1	Plot 2-D, réponses en phase et amplitude de H_{ap}	27
A.3.2	Plot 3-D représentation de l'amplitude de $H_{cf}(z)$ en fonction de la fréquence et dépendant du paramètre f_b ou d	29
	Références	30

1 Introduction

L'idée de ce projet vient d'une commune envie d'approfondir le concept de réverb numérique avec une approche un peu *programmation*. En commençant nos recherches nous nous rendons vite compte qu'il existe un code C++ d'un modèle de réverbération algorithmique légué au domaine public par Jazar en 2000. Ce code, que son développeur nomme sobrement Freeverb sera ensuite largement utilisé dans de nombreux plug-ins et s'impose comme étant un incontournable de la réverbération algorithmique sur les différents forum de développement. Il y a de ce fait pas mal de documentation qui traîne sur internet au sujet de la Freeverb. De plus, elle se base sur les travaux menés au siècle dernier par Schroeder et Moorer sur les réverbérations algorithmiques, elle n'est d'ailleurs qu'une mise en équation des principes développés dans leurs recherches. Ces recherches étant aussi dans le domaine public, on trouve pléthore de documentation à ce sujet ce qui nous paraissait être une condition nécessaire pour réaliser un compte rendu clair et complet. C'est donc sur la Freeverb que nous décidons d'axer ce dossier.

Dans un premier temps nous reviendrons sur les résultats de Manfred Schroeder et Ben Logan des Bell Laboratories sur les réverbérations algorithmique, avant d'étudier les améliorations apportées par James O. Moorer en travaillant avec les équipes de l'Ircam et de l'université de Stanford. Une fois le bloc théorique dégrossi, nous traiterons le cas spécifique de la FreeVerb, qui utilise les principes évoqués et rentrerons dans le détail du processing effectué par cet algorithme. Enfin nous finirons par une application de cet algorithme au sein d'un VST, nous prendrons pour exemple le plug-in *ReaVerbate* de Reaper et étudierons comment les paramètres de l'interface utilisateurs sont reliés aux variables du code source.

2 La première réverbération algorithmique

Manfred Schroeder et Ben Logan [1] sont les premiers à essayer de créer une réverb entièrement électronique capable de simuler des espaces acoustiques réalistes. Il publie en 1962 l'algorithme permettant d'obtenir "une réverb artificielle au son naturel". Leur travaux sont encore aujourd'hui une base pour la création de réverb algorithmique.

2.1 Une réverb incolore

Schroeder mesure qu'il faut 1000 échos par seconde pour avoir une réverbération tardive homogène sans flutter. Avant Schroeder les réverb électroniques étaient simplement constituées de filtres en peignes récursifs. Cependant cette méthode induisait, de manière inhérente au filtre en peigne, une coloration en fréquence dans la réverb. Schroeder a alors l'idée d'utiliser des filtres passe-tout (avec un gain unitaire pour toutes les fréquences) pour palier au problème de coloration. De plus comme ces filtres sont passe-tout, on peut en associer plusieurs en série pour multiplier la densité d'écho sans multiplier des potentiels défauts de coloration.

Si $g = 0,7$, le passe-tout décroît selon $-20 \log |g|$ donc pour un délai τ le temps de réverbération est :

$$T = \frac{60\tau}{-20 \log(0,7)} \iff T \approx 20\tau \quad (1)$$

Typiquement pour une réverb de 2s, il faudrait un τ de 100ms. Cependant un tel délai produit 10 échos par seconde ce qui est 100 fois moins que les 1000 souhaités. Il faut donc en aligner plusieurs en série et Schroeder choisit que chaque passe-tout successif ait une réponse impulsionnelle mesurant 1/3 du filtre précédant pour que ces délais soient des nombres premiers

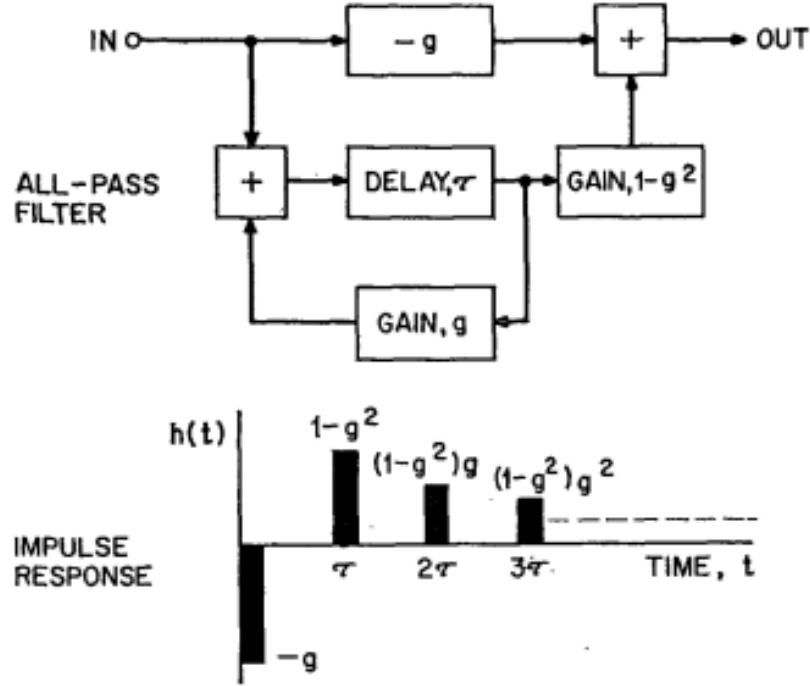


FIGURE 1 – Filtre passe-tout de Schroeder

entre eux. Il faut effectivement que les délais soit premiers entre eux pour éviter que certains échos s'ajoutent les uns sur les autres ce qui donnerait une décroissance non uniforme et moins dense.

$$\tau_i \approx \frac{100ms}{3^i}, i = 0, 1, 2, 3, 4 \quad (2)$$

Ainsi la densité d'écho de 5 passe-tout en série est de $81/\tau$ soit environ 810 échos par secondes.

2.2 Augmentation du réalisme

Cette réverbération fonctionne incommensurablement mieux que tout ce qui a été fait auparavant car celle-ci n'a plus les défauts de résonances et de flutter. Cependant pour atteindre un degré de réalisme supérieur, 3 aspects peuvent être poussés plus loin. On observe tout d'abord qu'il n'y a aucune gestion du pré-délai, la réverbération s'active après le temps τ_1 qui est de $100ms$. De plus Schroeder observe que dans une pièce, la réponse en fréquence d'une réverbération peu fluctuer de 10 à 40 dB et qu'il y a environ 15 pics de résonances dans chaque intervalles de 100Hz pour une réverbération de 1s. Pour implémenter ces deux aspects, Schroeder insert, en amont des filtres passe-tout, des filtres en peignes pour créer des résonances au sein de la réverbération (ce qui nous paraît tout à fait logique lorsque l'on sait qu'une réflexion sur un mur revient à filtrer en peigne le son).

Pour assurer suffisamment de densité, il faut en mettre assez en parallèles avec des délais différents et premier entre eux, celui avec le délai le plus court définira le pré-délai. En ce qui concerne les gains des filtres en peignes on réutilise la même formule citée plus haut tel que T soit le temps de réverbération souhaité :

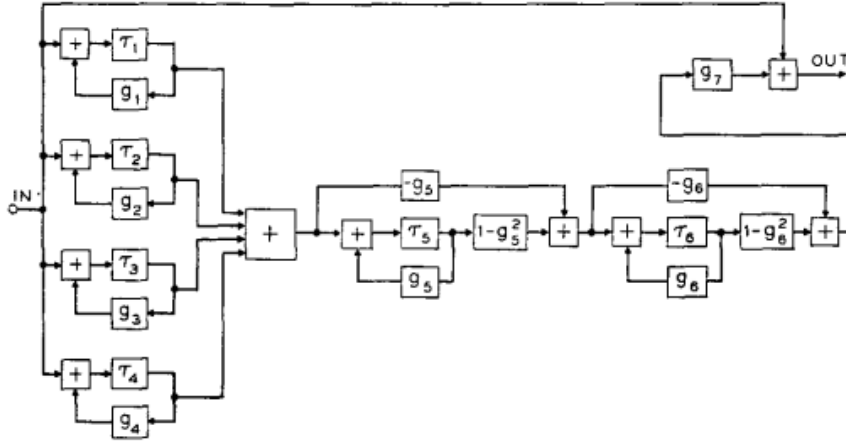


FIGURE 2 – Schéma bloc de la réverb de Schroeder utilisant 4 filtres en peignes et 2 passe tout

$$T = \frac{60\tau_n}{-20 \log |g_n|} \quad (3)$$

Schroeder rend également le temps de réverbération de sa réverb dépendant de la fréquence comme c'est le cas dans les salles de concerts avec un $Tr60$ plus grand pour les basses fréquences. Pour cela il ajoute simplement des filtres RC dans les boucles de feedback de ses filtres en peignes ce qui revient à atténuer les aigües dans les rétroactions.

Enfin, Schroeder à l'idée d'une réverbération immersive tridimensionnelle. Pour l'écoute multicanales, il suggère l'utilisation d'une matrices de résistance qui prendrai la sortie de chaque comb filter en phase et en opposition de phase. Cela permettrai d'alimenter jusqu'à 16 hauts parleurs et donner une réverb très diffuse.

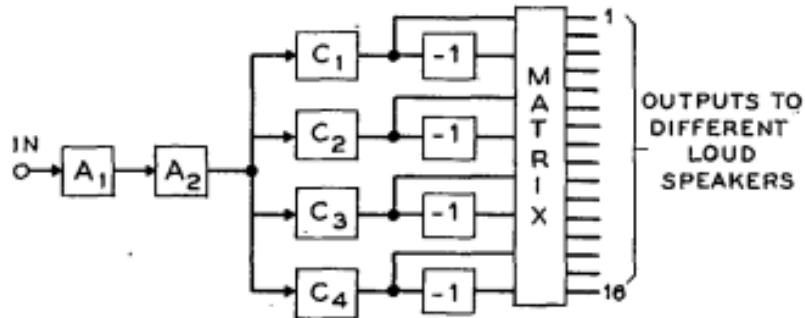


FIGURE 3 – réverb de schroeder connecté à la matrice de sortie alimentant 16 enceintes

3 Les améliorations de Moorer

James O. Moorer dans ses recherches avec l'Ircam et l'université de Stanford, va améliorer les travaux de Schroeder toujours dans un but de réalisme.

3.1 Les défauts du réverbérateur de Schroeder

Suites à de nombreux tests d'écoute James Moorer et ses équipes s'aperçoivent que la réverb de Schroeder souffre de plusieurs défauts. Le principal est que le decay est très métallique d'autant plus que les temps de réverbération sont long. Egalement, tout essai de réverbérer un son impulsif, comme une batterie par exemple, résulte en des échos flottants plutôt qu'en une réverbé fluide.

3.2 Vers une réverb plus réaliste

Pour Moorer [2] le principal défaut du réverbérateur de Schroeder est qu'il n'implémente pas de filtre (bien que Schroeder en fasse mention à la fin de son papier). L'absorption des hautes fréquences par l'air n'est pour lui pas un élément négligeable étant donné qu'il mesure dans certaine salle de concert comme au Boston Concert Hall que celle-ci peut atteindre 40 dB à 4 kHz. Il crée alors deux nouveaux schémas blocs (ceux utilisés dans la freeverb). Il rajoute un filtre du premier ordre dans la boucle de feedback du comb filter de Schroeder et crée un filtre all-pass en alliant un combfilter feedforward et un combfilter feedback (ces deux schémas seront étudiés plus en détails lors de l'analyse de la freeverb). Les coefficients du filtre passe bas sont données pour approcher les mesures réelles relatives à l'humidité, la température et la pression atmosphérique faites par Kuttruff.

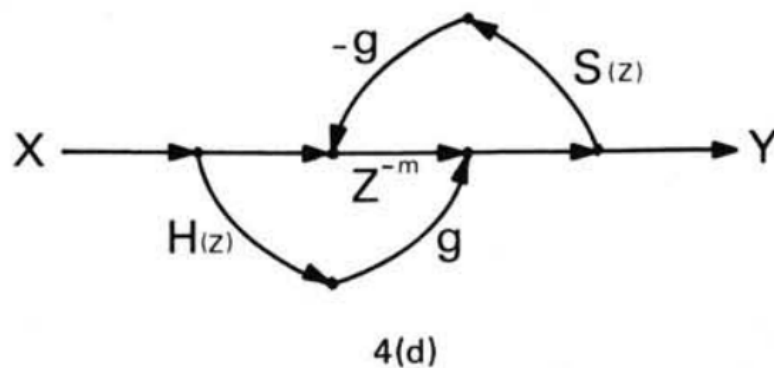
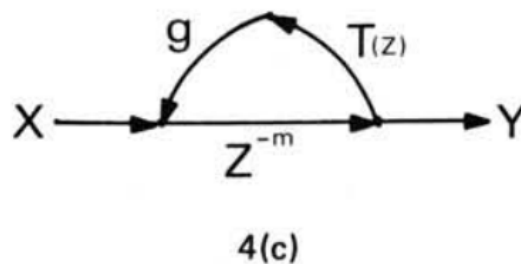


FIGURE 4 – Schéma du Comb filter filtrer et du All-pass filtrer créé par Moorer

Les fonctions de transferts du comb filter est donc :

$$Y(z) = z^{-m}(X(z) + gT(z)Y(z)) \quad (4)$$

Soit :

$$Y(z)(1 - gz^{-m}T(z)) = z^{-m}X(z) \quad (5)$$

$$\Leftrightarrow \frac{Y(z)}{X(z)} = \frac{z^{-m}}{1 - gz^{-m}T(z)} \quad (6)$$

$$\Leftrightarrow H_{fbcf}(z) = \frac{z^{-m}}{1 - gz^{-m}T(z)} \quad (7)$$

Ce filtre est appelé filtre en peigne à cause de sa réponse en fréquence. Ici l'entrée est en plus delayée de m samples et il y a un filtre dans la rétroaction de fonction de transfert $T(z)$.

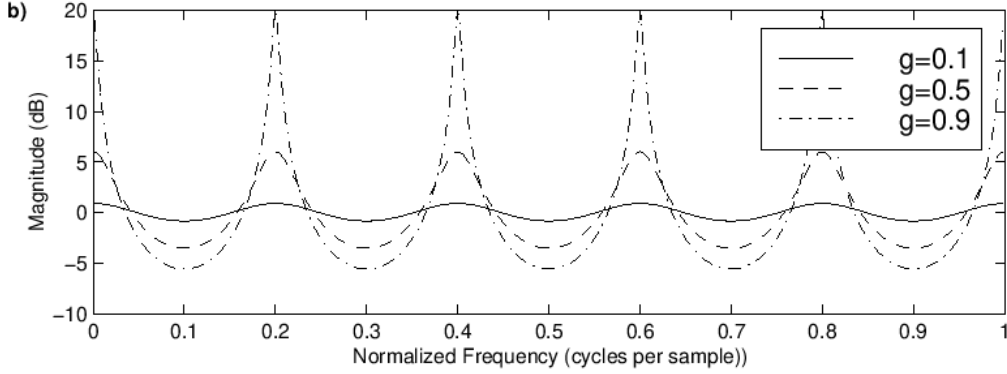


Figure 2.26: Amplitude response of the *feedback comb-filter* $H(z) = 1/(1 - gz^{-M})$ (Fig.2.24 with $b_0 = 1$ and $-a_M = g$) with $M = 5$ and $g = 0.1, 0.5$, and 0.9 . a) Linear amplitude scale. b) Decibel scale.

FIGURE 5 – Gain en dB du filtre en peigne à feedback [3]

Pour le passe-tout, sa fonction de transfert est :

$$Y(z) = z^{-m}X(z) + gH(z)X(z) - gS(z)z^{-m}Y(z) \quad (8)$$

Soit :

$$Y(z)(1 + gz^{-m}S(z)) = X(z)(gH(z) + z^{-m}) \quad (9)$$

$$\Leftrightarrow \frac{Y(z)}{X(z)} = \frac{gH(z) + z^{-m}}{1 + gz^{-m}S(z)} \quad (10)$$

$$\Leftrightarrow H_{ap}(z) = \frac{gH(z) + z^{-m}}{1 + gz^{-m}S(z)} \quad (11)$$

Ce filtre passe-tout est constitué d'un feedforward combfilter et d'un feedback combfilter et a un gain unitaire quand $|gH(z)| = |gS(z)|$.

L'implémentation de ce filtre produit bien une atténuation de l'aspect métallique de la queue présent chez Schroeder. Ça a aussi comme effet positif que la réverbère soit plus robuste au son impulsif en masquant les échos ainsi qu'une fusion plus grande des decays des comb filters. La configuration utilisé par Moorer est de 6 comb filters filtrés et un all-pass non filtré car il remarque que filtrer le passe-tout ou non ne fait aucune différence. Ainsi $|gH(z)| = |gS(z)| = 0$ et :

$$H_{ap}(z) = \frac{g + z^{-m}}{1 + gz^{-m}} \quad (12)$$

D'après lui, une configuration avec plus de comb filters ou de passe-tout n'ajoutera pas de différence audible.

3.3 Un problème inhérent à la conception du réverbérateur

Bien que ces améliorations produisent des effets positifs, Moorer est conscient que quel que soit le degré de perfectionnement du réverbérateur, il ne correspondra jamais au pattern de réflexion d'une salle réelle. Tout simplement car les réflexions dans une pièce n'arrivent pas espacées par un temps régulier comme c'est le cas pour les réverbérateurs de Schroeder et de Moorer.

Schroeder publie en 1969 un papier expliquant grâce au Ray tracing [4] comment reproduire le phénomène acoustique réel. Il faudrait pouvoir modéliser chaque premières réflexions puis ensuite le champ tardif homogène. Pour réaliser cela, les premières réflexions peuvent être modéliser par un filtre FIR et le champ tardif par un réverbérateur standard comme celui de Schroeder ou de Moorer. Tout d'abord on simule les n premières réflexions avec, pour chacune, un délai et un gain séparé. Cela occupe généralement les 40 à 80 premières ms.

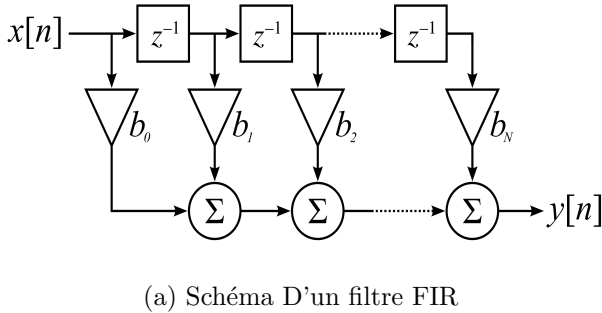


Table 3. Tap times (in seconds) and gains for a reasonable-sounding seven-tap FIR section for reverberation. Tap number corresponds to the subscript of the coefficient (a_j) and delay (m_j) shown in Figure 12.

TAP	TIME	GAIN
0	0	1.00
1	.0199	1.02
2	.0354	.818
3	.0389	.635
4	.0414	.719
5	.0699	.267
6	.0796	.242

Tap times (in seconds) and gains for a reasonable-sounding 19-tap FIR section for reverberation. These were taken from a highly idealized geometric simulation of the Boston Symphony Hall.

TAP	TIME	GAIN
0	0	1.0
1	.0043	.841
2	.0215	.504
3	.0225	.491
4	.0268	.379
5	.0270	.380
6	.0298	.346
7	.0458	.289
8	.0485	.272
9	.0572	.192
10	.0587	.193
11	.0595	.217
12	.0612	.181
13	.0707	.180
14	.0708	.181
15	.0726	.176
16	.0741	.142
17	.0753	.167
18	.0797	.134

(b) Valeurs des délais et des gains de chaque retards pour les 2 FIR réalistes de Moorer

FIGURE 6 – Filtre à Réponse Impulsionnelle Fini pour modéliser les premières réflexions

On envoie ensuite le signal d'entrée, delayé du temps du FIR, dans un réverbérateur comme celui de Schroeder. Pour Moorer le réverbérateur de champ tardif qui fonctionne le mieux est le sien évoqué plus haut.

Cependant Moorer s'aperçoit que simuler une pièce par cette méthode n'est aucunement réaliste ce qui signifie qu'une de leur hypothèse est erronée. Après observations des réponses impulsionnelles de salles enregistrées par Schroeder et Gottlob, il s'aperçoit qu'il a ignoré l'effet de diffusion créé par les murs non plat mais irrégulier. Ainsi à l'intérieur des premières réflexions distinctes, se trouvent une multitude de réflexions au gain très faibles causées par les petites irrégularités des murs dans les concert Hall. Pour pallier à ce problème, Moorer propose d'envoyer en plus du son direct, chaque délais du FIR dans dans le réverbérateur. Ainsi, la densité des premières réflexions est amplifié par les délais du réverbérateur.

Les délais D1 et D2 sont ajustés tels que le premier écho du réverbérateur arrive en en même

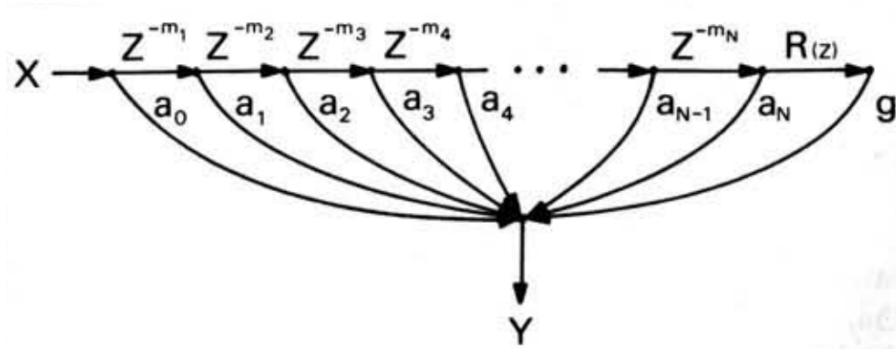


FIGURE 7 – Réverbe pensé par Schroeder alliant un filtre FIR et un réverbérateur (1970)

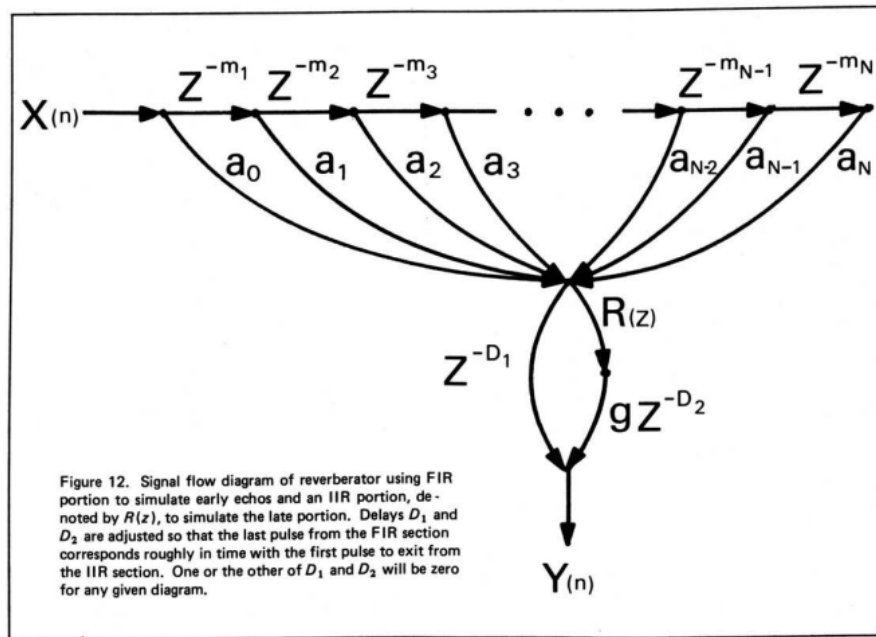


FIGURE 8 – Réverb pensé par Moorer alliant un FIR et un réverbérateur

temps que le dernier délais du FIR. Grâce à ce changement, la réverbe se rapproche de l'acoustique d'une vraie salle de concert.

Cependant avec les processeurs de l'époque, et même avec la FFT, réaliser un filtre FIR pour simuler les 80 premières ms de la salles est hors de portée (un FIR revient à de la convolution pure). De nombreuses méthodes ont été essayé pour synthétiser réalistiquement les premières réflexions : utiliser les résultats d'une simulation géométrique voir choisir des nombres aléatoires pour déterminer la position et l'amplitude. Mais cela à toujours résulté en la création involontaire de gros filtres en peignes et cela même si la position des réflexions est choisi comme un nombre premier du sample d'origine (cela s'explique car la somme ou la différence de 2 nombres premier est toujours divisible par 2). De manière itérative, Moorer et ses équipes ont réussi à isoler de deux tap-delays qui sonne réalistiquement bien.

4 La Freeverb, un modèle de réverbération plutôt classique

La Freeverb est basée sur le fonctionnement d'un diffuseur de Schroeder. Le code fait notamment appel à deux modèles de filtres qui sont les murs porteurs de cette réverbération. Ces filtres sont le **combfilter** (filtre en peigne) et le **allpassfilter** (filtre passe-tout). La FreeVerb utilise 8 filtres en peigne et 4 filtres passe-tout, chacun avec des délais différents afin de créer cette impression de flou propre à l'effet de réverbération.

4.1 Le combfilter

Le combfilter ou filtre en peigne est appelé ainsi car la partie réelle de sa fonction de transfert forme un peigne de sorte que certaines fréquences passent de manière inchangées et d'autres sont complètement atténuées et ceci alternativement formant des dents comme sur un peigne.

4.1.1 Processing audio

Dans cette partie nous allons voir au sein du code source en c++ comment sont créés les filtres en peignes. Ci-dessous le processing qui est effectué chaque fois que la fonction combfilter est appelée dans le code.

```
inline float comb::process(float input)
{
    float output;

    output = buffer[bufox];
    undenormalise(output);

    filterstore = (output*damp2) + (filterstore*damp1);
    undenormalise(filterstore);

    buffer[bufox] = input + (filterstore*feedback);

    if(++bufox>=bufsize) bufox = 0;

    return output;
}
```

FIGURE 9 – Définition de la fonction comb.process

Nous trouvons aussi 3 facteurs que sont **damp1**, **damp2** et **feedback**. Il s'agit de multiplications que l'on applique sur les échantillons afin de gérer la quantité de feedback, c'est-à-dire la quantité du n-ième échantillon dans le n+1-ième.

4.1.2 Implémentation fonctionnelle

Afin d'y voir un peu plus clair on se propose de faire l'implémentation fonctionnelle équivalente au code que l'on vient de voir.

4.1.3 Fonction de transfert $H_{cf}(z)$

On sait que la fonction de transfert peut s'exprimer dans l'espace des fréquences comme le rapport de la sortie sur l'entrée :

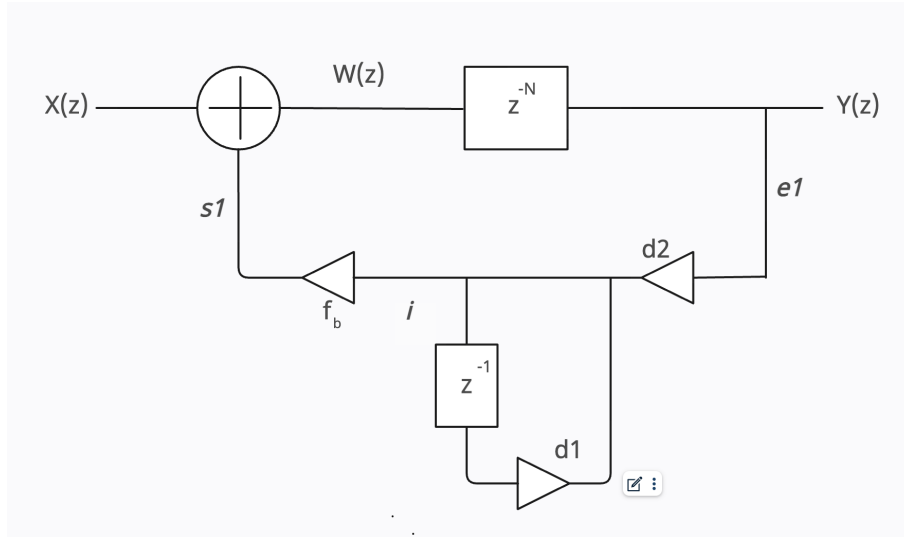


FIGURE 10 – Implémentation fonctionnelle du combfilter

$$H(\omega) = \frac{Y(\omega)}{X(\omega)} \quad (13)$$

Donc tâchons de déterminer la fonction de transfert de ce filtre passe tout, on garde à l'esprit qu'il y'a une équivalence entre $H(z)$ et $H(\omega)$ lorsque z est restreint au cercle unité, $z \in \mathcal{C}(0, 1)$.

Pour des raisons de simplicité nous appellerons $G(z)$ la fonction équivalente à l'ensemble de la boucle de rétroaction, c'est-à-dire l'ensemble des opérations subies par le signal entre e_1 et s_1 . En somme :

$$G(z) = \frac{s_1(z)}{e_1(z)}$$

Ceci nous permet d'écrire :

$$Y(z) = (X(z) + G(z)Y(z)) z^{-N} \quad (14)$$

$$\iff Y(z)(1 - G(z)z^{-N}) = X(z)z^{-N} \quad (15)$$

$$\iff \frac{Y(z)}{X(z)} = \frac{z^{-N}}{1 - G(z)z^{-N}} \quad (16)$$

Tentons maintenant de trouver l'expression de $G(z)$

Considérons l'équation aux différences au point i .

On pose $d_1 \triangleq \text{damp1}$ et $d_2 \triangleq \text{damp2}$

$$\begin{aligned} i(z) &= d_2 e_1(z) + d_1 z^{-1} i(z) \\ \iff i(z) (1 - d_1 z^{-1}) &= d_2 e_1(z) \\ \iff i(z) &= \frac{d_2 e_1(z)}{1 - d_1 z^{-1}} \\ \iff \frac{i(z)}{e_1(z)} &= \frac{d_2}{1 - d_1 z^{-1}} \end{aligned}$$

$$\text{Or} \quad s_1(z) = f_b i \quad (17)$$

$$\text{On déduit} \quad \frac{s_1(z)}{e_1(z)} = \frac{f_b d_2}{1 - d_1 z^{-1}} = G(z) \quad (18)$$

Or dans le bloc au sein du quel sont définies les fonctions relatives au combfilter, on trouve dans le code de la Freeverb la fonction **setdamp** qui relie **damp1** et **damp2**.

```
void comb::setdamp(float val)
{
    damp1 = val;
    damp2 = 1-val;
}
```

FIGURE 11 – La fonction setdamp

On appelle donc $d_1 \triangleq d$ et $d_2 \triangleq 1 - d$

d est le damping et signifie amortissement en français, nous reviendront sur son utilité plus tard.

On reprend 18 avec ces nouvelles notations :

$$G(z) = f_b \frac{1 - d}{1 - dz^{-1}} \triangleq f_b T(z) \quad (19)$$

Où $T(z)$ est un filtre passe bas récursif du premier ordre. En réinjectant 19 dans 16 on obtient la forme développée de notre chère fonction de transfert.

$$H_{cf}(z) = \frac{Y(z)}{X(z)} = \frac{z^{-N}}{1 - f_b \cdot z^{-N} \underbrace{\left(\frac{1 - d}{1 - dz^{-1}} \right)}_{T(z)}} \quad (20)$$

On retombe sur la fonction de transfert 7 décrite dans les travaux de Moorer, avec $T(z)$ un filtre passe-bas du premier ordre et $f_b \triangleq g$, g étant selon Moorer le gain total de la rétroaction.

Ce que nous dit cette fonction de transfert 20 c'est qu'il s'agit là d'un feedback combfilter dont la contre-réaction est filtrée par le filtre $T(z)$, nous appelons donc le filtre de l'équation 20 un filtered feedback combfilter. La Freeverb met donc en pratique le phénomène mis en lumière par Moorer 3.2 selon lequel l'absorption des hautes fréquences par l'air dans les salles de concerts n'était pas négligeable. Ainsi dans ce filtered feedback combfilter les hautes fréquences étant filtrées dans la contre-réaction et donc de moins en moins réinjectées au fur et à mesure des itérations vont s'atténuer plus rapidement que les basses fréquences.

4.2 Le allpass filter

Nous avons vu le premier type de filtre, regardons maintenant plus en détail le second type de filtre utilisé dans cette reverb : le filtre passe-tout.

Sur le papier le filtre passe tout, n'a aucun impact sur le spectre du signal il est supposé parfaitement plat en regard de la partie réelle de sa fonction de transfert. En revanche il va générer des micros retards, des effets de phase qui vont flouter le signal, on retrouve de manière quasi-systématique ce type de filtre dans les reverbs algorithmique.

4.2.1 Processing audio

Permettons nous de jeter un oeil à l'implémentation C++ du allpass filter. Il s'agit ici de la fonction qui fait le processing afin de générer un filtre passe tout.

```
inline float allpass::process(float input)
{
    float output;
    float bufout;

    bufout = buffer[bufidx];
    undenormalise(bufout);

    output = -input + bufout;
    buffer[bufidx] = input + (bufout*feedback);

    if(++bufidx>=bufsize) bufidx = 0;

    return output;
}
```

FIGURE 12 – Définition de la fonction allpass.process

Ici nous avons donc un tableau c'est-à-dire une série de nombre appelé **buffer** c'est évidemment un buffer dont la taille varie en fonction des préférences de l'interface audio (64-128-256-512-1024 etc). Ce tableau est parcouru par la grandeur **bufidx** (*=buffer index*) qui augmente de 1 à chaque itération. A chaque itération on commence par stocker la valeur du buffer pour l'indice **bufidx** dans la variable **bufout**. Ensuite **buffer[bufidx]** prend une nouvelle valeur avec la contribution de la valeur **feedback**. Enfin on s'assure que l'index qui parcourt le buffer ne dépasse pas la taille du buffer, sinon on le remet à 0, cela veut dire qu'on traite un nouveau buffer.

4.2.2 Implémentation fonctionnelle

Voici le schéma équivalent au code explicité juste au dessus. 13

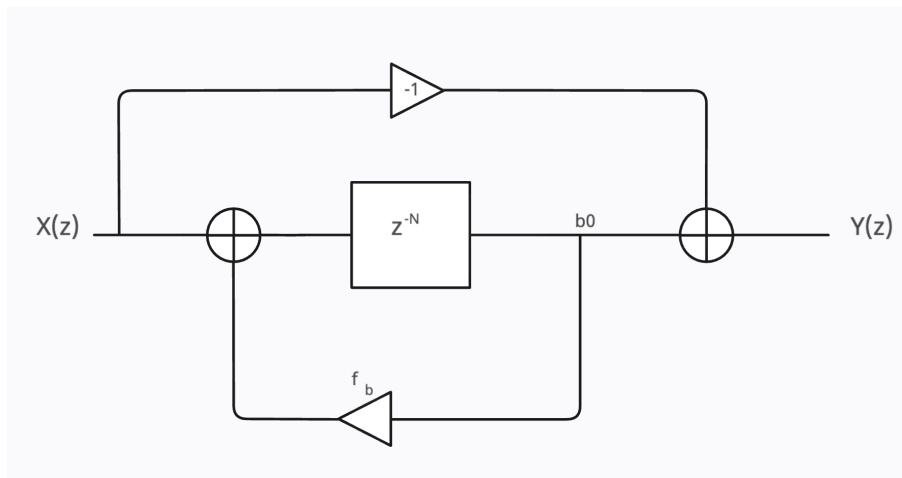


FIGURE 13 – Implémentation numérique du allpass filter

4.2.3 La fonction de transfert $H_{ap}(z)$

De manière analogue a ce qui a été fait précédemment, nous utilisons l'équation 13 pour en déduire l'expression de $H_{ap}(z)$

Au point b_0 le potentiel s'écrit d'après la figure 13 :

$$\begin{aligned}
 b_0 &= [X(z) + b_0(z)f_b]z^{-N} \\
 \Leftrightarrow \quad b_0(z)(1 - z^{-N}f_b) &= X(z)z^{-N} \\
 \Leftrightarrow \quad \frac{b_0(z)}{X(z)} &= \frac{z^{-N}}{1 - f_b z^{-N}}
 \end{aligned} \tag{21}$$

On reconnaît ici l'expression d'un **feedback combfilter**, en considérant comme on vient de le faire le signal seulement entre $b_0(z)$ et $X(z)$ on retrouve l'expression 20 avec ici $T(z) = 1$ c'est-à-dire équivalent à $d = 0$ un amortissement nul.

On peut faire de même pour l'expression de la sortie du filtre $Y(z)$

$$Y(z) = -X(z) + b_0(z) \tag{22}$$

Connaissant l'expression de b_0 on injecte 21 dans 22 et on obtient ce qui suit.

$$\begin{aligned}
Y(z) &= -X(z) + \frac{z^{-N}}{1 - f_b z^{-N}} X(z) \\
\iff \frac{Y(z)}{X(z)} &= -1 + \frac{z^{-N}}{1 - f_b z^{-N}} \\
\iff \frac{Y(z)}{X(z)} &= \frac{-1 + f_b z^{-N} + z^{-N}}{1 - f_b z^{-N}} \\
\iff \frac{Y(z)}{X(z)} &= \frac{-1 + (f_b + 1)z^{-N}}{1 - f_b z^{-N}} \\
\iff \frac{Y(z)}{X(z)} &= \underbrace{\frac{1}{1 - f_b z^{-N}}}_{\text{feedback combfilter}} \underbrace{(-1 + (f_b + 1)z^{-N})}_{\text{feedforward combfilter}} = H_{ap}(z)
\end{aligned}$$

Cette fois on ne retombe pas sur la forme d'un passe-tout selon Moorer 12. On se propose d'observer la réponse en amplitude de ce filtre avec une représentation graphique. 14

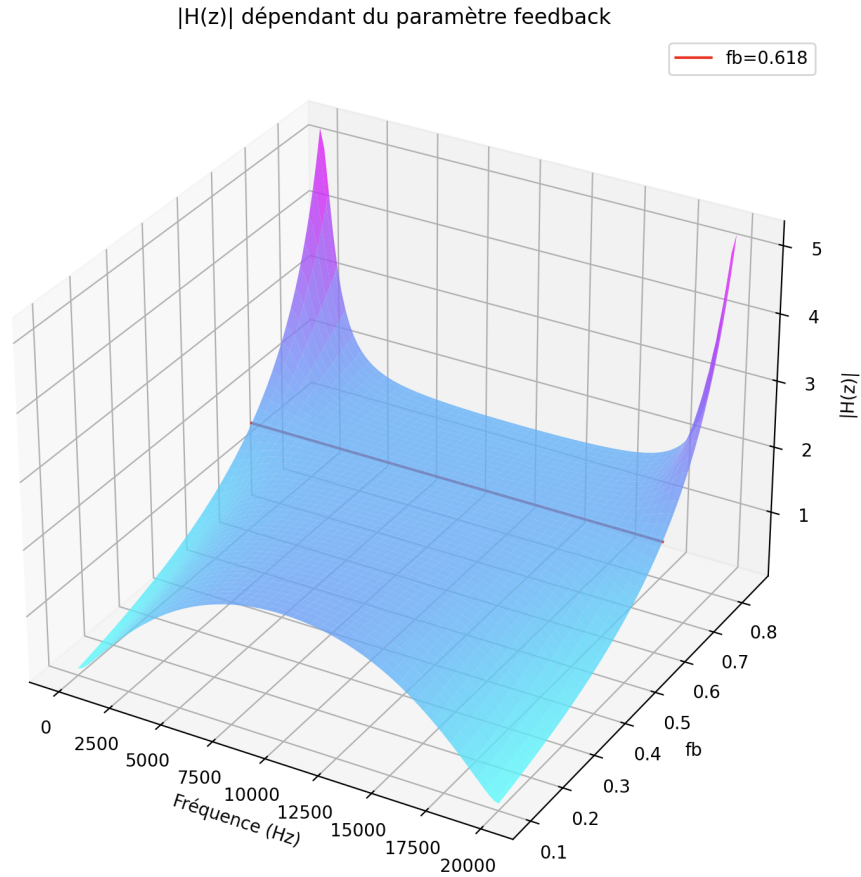


FIGURE 14 – Diagramme de Bode du filtre passe tout pour différentes valeurs de feedback fb , $N=1$

On remarque qu'à l'ordre 1, il y a bien un moment où le filtre est un vrai passe tout avec un gain constant sur l'ensemble de la bande de fréquence, cette valeur est mise en évidence sur

la figure 14 et équivaut à $f_b = 0.618$ l'inverse du nombre d'or. En 15 On fait un zoom sur la tranche de 14 en $f_b = 0.618$

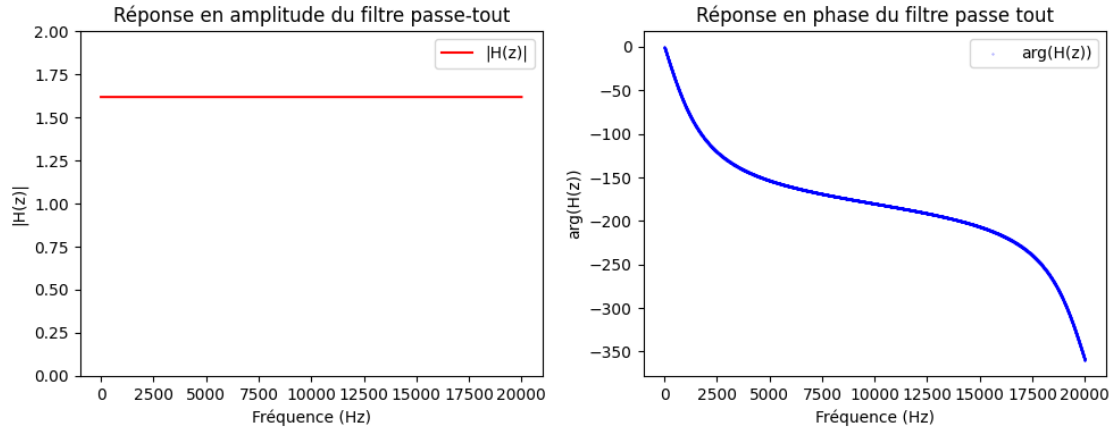


FIGURE 15 – Diagramme en amplitude et en phase de $H_{ap}(z)$ pour $N = 1$ et $f_b = 0.618$

On a donc un déphaseur pur qui retarde les fréquences aigües par rapport aux graves sans atténuer ou amplifier aucune fréquences.

Seulement dans la Freeverb la valeur du **feedback** est fixée à 0.5 et pas 0.618.

De plus ce filtre n'est plus un all-pass filter dès lors que l'ordre du filtre augmente. En accord avec 16, il n'existe aucune valeur de feedback permettant de créer un déphaseur pur pour $N > 1$. Peut-être qu'il est possible d'améliorer le code à ce niveau là.

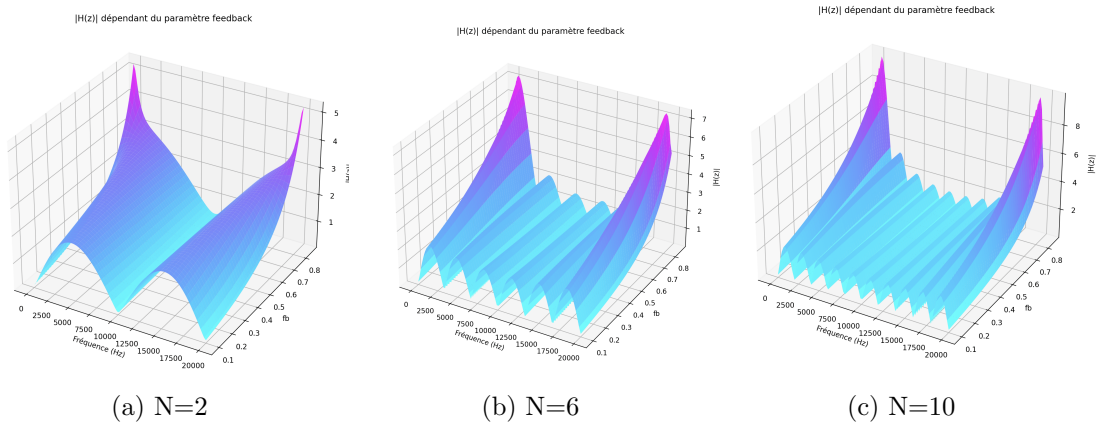


FIGURE 16 – Diagramme de Bode du filtre "passe-tout" avec en paramètre le feedback, pour différentes valeurs de N

Pour autant on ne peut pas avancer avec certitude ce que nous venons de dire, en effet on suspecte en 16 une erreur lors de l'affichage que nous n'avons à ce jour pas résolue. Les fréquences extrêmes $f = 0\text{Hz}$ et $f = 20\text{kHz}$ sont systématiquement amplifiées ce qui ne devrait pas être le cas.

4.3 Tuning de la reverb

Ce que nous appelons "*tuning*" est le fait de choisir des gains et des délais adéquats pour que l'effet de réverbération soit réaliste. On se base pour cela sur les travaux de M. Schroder et B. Logan détaillés dans le 2. Le créateur de la Freeverb, Jezar affirme avoir passé du temps sur cette étape d'accordage de la reverb, c'est quelque chose qui se fait à l'oreille, à tatons, et Schroder lui-même déclare que c'est bien là tout le coeur du problème si on veut une reverb réaliste.

Nous pouvons observer sur la figure 22 que pour chaque filtre il y'a deux buffer, un pour le canal droit et l'autre pour le canal gauche. Ce code contient les informations de délais pour chaque filtre, ici Jezar semble avoir respecté l'idée de Schroeder selon laquelle les délais doivent être premiers entre eux, seulement ce n'est pas le cas pour tous, par exemple 441 et 225 ainsi que 1116 et 1188 ne sont pas premiers entre eux. Jezar propose une valeur de 23 samples pour la grandeur stereospread, ainsi nous pouvons constater qu'à chaque fois, le retard droit est plus long de 23 samples. Ceci a pour but de créer un effet de stéréo en augmentant les différences entre le signal de droite et le signal de gauche.

4.4 Architecture globale de la Freeverb

Comme dit précédemment la Freeverb est une reverb basée sur le modèle de Schroeder, on retrouve bien des filtres en peigne en parallèles suivis de filtres passe tout disposés en série. Il suffit pour s'en convaincre de jeter un bref coup d'oeil à la figure 17.

Cette portion de code 17 consiste en une boucle *while*. L'initialisation des variables se fait à 0 pour les sorties et on fait une sommation des deux canaux d'entrée dans la variable **input**. Ensuite nous avons une boucle *for* pour nos 8 filtres en peignes, en effet ici **numcombs** = 8. Les lignes 87 et 88 sont l'affectation des nouvelles valeurs aux sorties, le détail de ces opérations est dans le 4.1.3.

On rappelle qu'en C++ :

$$\text{outL} += \text{combL}[i].\text{process}(\text{input}) \iff \text{outL} = \text{outL} + \text{combL}[i].\text{process}(\text{input})$$

Il est intéressant de constater qu'à ce stade **outL** et **outR** ne diffèrent que par la longueur de **combR** et **combL**, en accord avec ce que nous avons dit en 4.3 sur la stereospread, car la fonction **process** est identique sur les deux canaux et surtout **input** est une sommation brutale des canaux droite et gauche.

Il suit une autre boucle *for* cette fois-ci dédiée au processing des 4 allpass filters (car **numallpass** = 4), ces opérations ont été détaillées dans le 4.2.3.

Enfin nous trouvons les lignes 99 et 100 qui seront les sorties finales de la reverb, ici on applique une *matrice de spatialisation* dont le but est de décorrélérer les canaux droite et gauche (nous développerons cela dans le 5.1), et une opération portant sur l'entrée de la reverb permettant de gérer le rapport dry/wet.

Le dry/wet est un des paramètres que l'on peut régler, mais il y en a bien d'autres, nous l'avons dit, ce qui fait la force des reverbs algorithmique dans la production audiovisuelle c'est leur forte adaptabilité, c'est-à-dire qu'on peut modeler le son de reverb désiré si on sait sur quels paramètres influencer. Maintenant que nous avons compris l'architecture générale de cet algorithme, tentons de voir à quel point celui-ci est modulable.

```

74 void revmodel::processreplace(float *inputL, float *inputR, float *outputL,
75 float *outputR, long numsamples, int skip)
76 {
77     float outL, outR, input;
78
79     while(numsamples-- > 0)
80     {
81         outL = outR = 0;
82         input = (*inputL + *inputR) * gain;
83
84         // Accumulate comb filters in parallel
85         for(int i=0; i<numcombs; i++)
86         {
87             outL += combL[i].process(input);
88             outR += combR[i].process(input);
89         }
90
91         // Feed through allpasses in series
92         for(i=0; i<numallpasses; i++)
93         {
94             outL = allpassL[i].process(outL);
95             outR = allpassR[i].process(outR);
96         }
97
98         // Calculate output REPLACING anything already there
99         *outputL = outL*wet1 + outR*wet2 + *inputL*dry;
100        *outputR = outR*wet1 + outL*wet2 + *inputR*dry;
101    }
102 }

```

FIGURE 17 – Processing général de la Freeverb.

5 ReaVerbate, un VST utilisant le processing de le Freeverb

Dans cette partie nous nous baserons sur l'interface utilisateur de le ReaVerbate et tenterons de relier les paramètres à la manière dont ils modifient les équations dans le code.

5.1 La stereo width ou largeur stéréo

La variable **dry** est directement reliée à l'interface utilisateur, une partie du code située en annexe 23 consiste à mettre à jour les paramètres dès qu'ils sont changés par l'utilisateur. Plus la variable **dry** est grande plus on injecte de son direct dans la sortie et donc moins on a l'effet de réverbération.

Au contraire la variable **wet** fait tout l'inverse, notons qu'il n'y a dans la *Freeverb* pas de liens entre **dry** et **wet** là où certains plug-ins définissent l'un par rapport à l'autre, généralement de sorte que $wet = 1 - dry$. Nous ne serons donc pas surpris de trouver deux potentiomètres (un pour le wet, un pour le dry) dans les VST utilisant la Freeverb, c'est par exemple le cas de *ReaVerbate* 18 de Reaper auquel le développeur Cockos a rajouté quelques fonctions.

Revenons maintenant à notre *matrice de spatialisation* et aux lignes 99 et 100 de 17. Une partie du canal droit post reverb est injecté dans le canal gauche et vice-versa. Les variables **wet1** et **wet2** sont reliés aux variables **wet** et **width** selon la figure 19

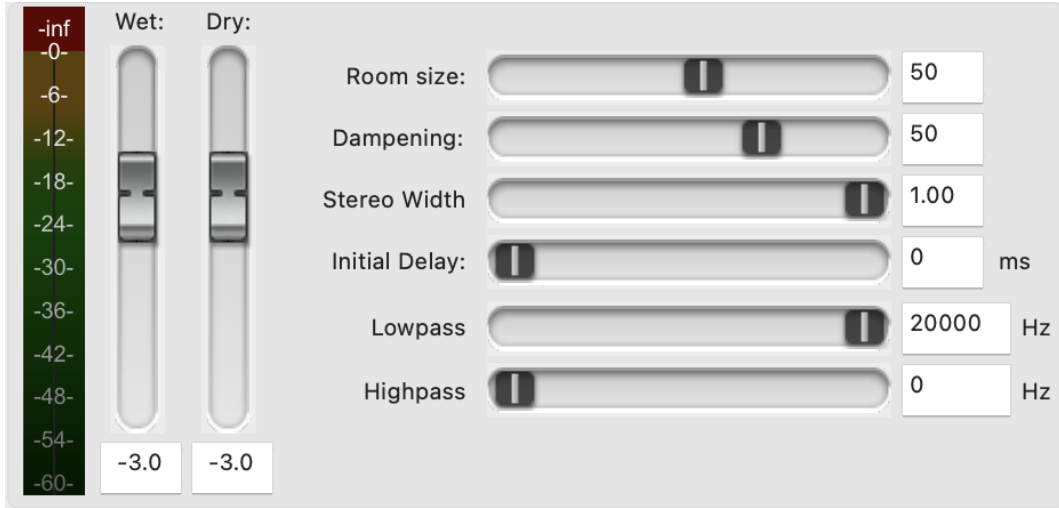


FIGURE 18 – Interface utilisateur de Reaverbate, un VST utilisant le code source Freeverb

```
wet1 = wet*(width/2 + 0.5f);
wet2 = wet*((1-width)/2);
```

FIGURE 19 – définition de wet1 et wet2

On montre facilement (cf A) que

$$L = \frac{W_1 - W_2}{W_1 + W_2} \quad (23)$$

Où L est la largeur stéréo (=stereowidth) de la reverb.

On peut regarder ce qui se passe sur **outputL** et **outputR** pour certaines valeurs de L en considérant un son full wet c'est-à-dire avec **dry**= 0. Sur la figure 18, **stereowidth** semble varier entre -1 et 1, c'est-à-dire $L \in [-1; 1]$

$$— L = 0 \iff W_1 = W_2$$

Dans ce cas, en reprenant les lignes 99 et 100 de la figure 17, **outputL** = **outputR**. On a alors une reverb mono avec une latéralisation nulle mais qui va tout de même apporter un effet de placement et de profondeur.

$$— L = 1 \iff W_1 - W_2 = W_1 + W_2 \iff W_2 = 0 \text{ et } W_1 = W$$

Dans ce cas **outputL** = **outL** × **wet1** et **outputR** = **outR** × **wet1**

On a une décorrélation maximale, pas de réinjection de la droite dans la gauche ou inversement et donc une grande impression de largeur.

$$— L = -1 \iff W_2 - W_1 = W_1 + W_2 \iff W_1 = 0 \text{ et } W_2 = W$$

Dans ce cas **outputL** = **outR** × **wet2** et **outputR** = **outL** × **wet2**

Ici encore on a une décorrélation maximale entre canal droite et canal gauche, mais on a cette fois uniquement la réinjection du canal droit dans le canal gauche et uniquement la réinjection du canal gauche dans le canal droit, on inverse de fait l'image par rapport au cas $L = 1$.

En jouant sur la valeur de **stereo width** on influe sur la largeur de la stéréo (logique!) En jouant en plus avec le niveau de **dry** et **wet** on influe sur le placement des sources dans la profondeur.

5.2 Le damping ou amortissement

Sur l'interface utilisateur 18 **damping** varie entre 0 et 100. Cependant il est probable que dans le code de la Freeverb, **damp** soit plutôt compris entre 0 et 1, car rappelons que d'après 11 :

$$\begin{cases} d_1 \triangleq d \\ d_2 \triangleq 1 - d \end{cases} \quad (24)$$

De sorte que $d_1 + d_2 = 1$ tout le temps, cette égalité permet d'avoir un gain global constant et de ne pas faire saturer l'entrée de l'AOP f

En regardant la figure 10 on comprend que $d_2 = 1 - d$ correspond à la quantité de l'échantillon précédent réinjecté dans le circuit, on s'attend à voir un filtrage en peigne d'autant plus violent que $d - 2$ est grand (et donc que $d = d_1$ est petit).

On rappelle ici la fonction de transfert du comb filter dont la démonstration est faite en 4.1.3 :

$$H_{cf}(z) = \frac{Y(z)}{X(z)} = \frac{z^{-N}}{1 - f_b z^{-N} \left(\frac{1-d}{1-dz^{-1}} \right)} \quad (25)$$

On regarde le comportement de H_{cf} pour différentes valeurs de d à f et z constants.

— $d = 0$

La fonction H_{cf} restreinte à $d = 0$ donne :

$$H_{cf}(z)|_{d=0} = \frac{z^{-N}}{1 - f_b z^{-N}}$$

On retombe sur le cas de 7 de la reverb de Moorer avec $T(z) = 1$ et $g \triangleq f_b$. En regardant la figure 5 on voit le profil de $|H_{cf}(z)|_{d=0}$ et on a un peigne d'autant plus creusé que $f_b \triangleq g$ est grand. C'est logique, plus on met de feedback plus le filtre en peigne sera violent.

— $d = 1$

La fonction H_{cf} restreinte à $d = 1$ donne :

$$H_{cf}(z)|_{d=1} = z^{-N}$$

Ce qui est plutôt normal, choisissant $d_1 = 1$ on a de fait $d_2 = 0$ et donc rien ne passe dans la rétroaction, on retrouve en sortie $Y(z)$ simplement le signal d'entrée $X(z)$ décalé de N samples.

— $d \in]0; 1[$

Grâce à la librairie *matplotlib* on peut tracer des graphes en 3-D sur Python permettant de faire apparaître le gain c'est-à-dire le module de la fonction de transfert sur l'axe vertical, la fréquence en abscisse et l'influence du paramètre d sur le dernier axe. Les calculs se feront avec une valeur de **feedback** $f_b = 0.84$ car c'est la valeur initiale de f dans le code de la FreeVerb

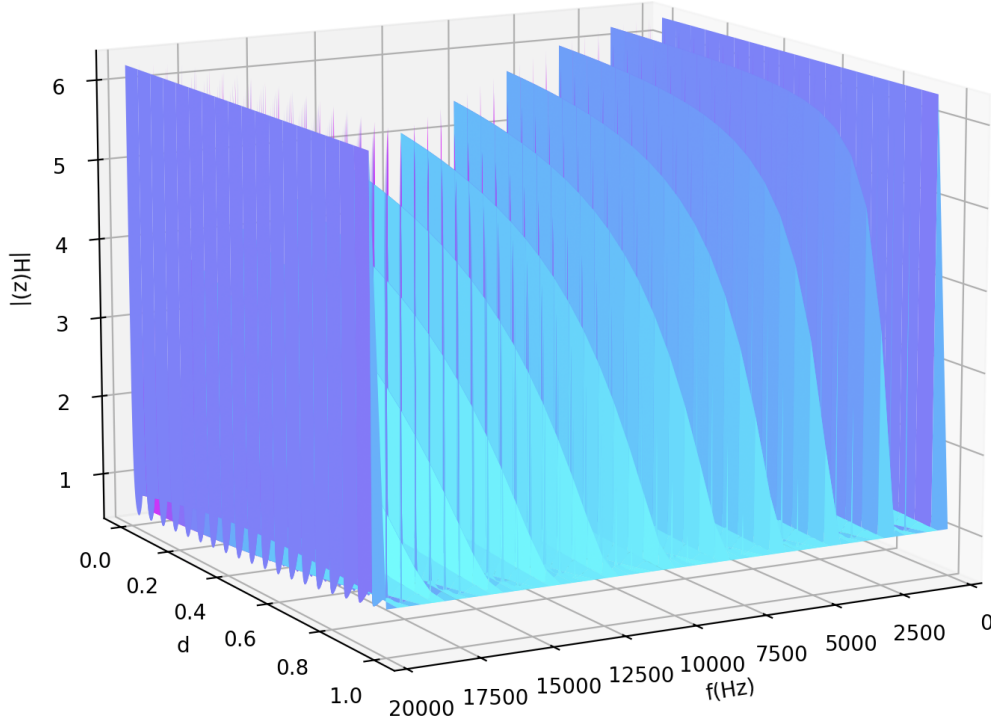


FIGURE 20 – Influence du paramètre damping sur $|H_{cf}(z)|$

Ce graphique est réalisé pour $f_b = 0.84$, la fréquence f est reliée à la variable z par la relation $z = e^{2\pi j f}$

On remarque que plus l'amortissement est grand moins il y a de rétroaction et donc moins il y a de filtrage en peigne, on tend vers un gain unitaire sur l'ensemble de la bande passante pour $H_{cf}(z)$. On constate également que plus la fréquence est élevée, plus le signal est filtré, c'est l'influence de notre fonction $T(z)$ dans la fonction de transfert du filtered feedback combfilter. De plus rappelons que dans 19 d est reliée à la fréquence de coupure du filtre, ceci aussi s'observe sur la figure ??, en effet les hautes fréquences sont déjà atténuées pour des petites valeurs de d alors qu'il faut attendre $d > 0.9$ pour faire chuter les graves.

Attention cependant, même si on a des raisons de penser que ce graphique est en partie correct, il n'en reste pas moins certainement en partie faux, en effet on devrait voir pour d qui tend vers 0 un vrai filtre en peigne non filtré, or ce n'est pas le cas. Nous admettons que la mise en schéma imparfaite de ces résultats est dommage et nous ne sommes pas contre une lumière pour nous éclairer là dessus (les codes sont en annexes).

Le paramètre de **damping** n'intervient dans la fonction *allpass :process*, nous n'étudierons donc pas l'influence de ce paramètre sur H_{ap}

En conclusion le damping va influencer la manière dont les premières réflexions seront plus ou moins chargées d'aigus. Ainsi un facteur de **damping** faible n'atténue pas les aigus quand ils rebondissent sur les murs, c'est l'équivalent d'une room avec des parois plutôt en verre ou en métal, tandis qu'une valeur grande va plutôt modéliser des murs en matériaux qui absorbent les hautes fréquences comme le bois ou la mousse.

5.3 Le feedback

Le paramètre de feedback est présent dans les deux implémentations des filtres 10 et 13 sous le sigle f_b , cependant dans le code, lorsque ce paramètre varie, seul les filtres en peignes sont mis à jour avec cette nouvelle valeur. Les filtres passe-tout restent identiques du début à la fin, cela veut dire que la partie diffus du champ réverbéré est toujours la même, cela est cohérent avec les observations de Moorer 3.2. Nous nous pencherons donc ici encore sur les fonctions de transfert des filtres en peigne uniquement.

Le **feedback** est intimement relié à la **roomsize** dans le code Freeverb, aussi par abus de langage nous utiliserons préférentiellement le terme feedback.

- Le cas particulier $f_b = 0$ La fonction $H_{cf}(z)$ restreinte à $f_b = 0$ donne :

$$H_{cf}(z)|_{f_b=0} = z^{-N} \quad (26)$$

Ce qui paraît logique dans le sens où $f_b = 0$ signifie aucune rétroaction, on retrouve donc simplement le signal d'entrée décalé de N samples.

Lorsque que le **feedback** est proche de 0, on considère qu'il est égal à zéro. En observant l'interface 18 que la Roomsize est cantonnée entre 23 et 98 ce qui est un agrandissement par 100 de l'intervalle $[0.23; 0.98]$ afin de rendre l'interface utilisateur plus agréable. $[0.23; 0.98]$ sont les ordres de grandeurs utilisés dans la Freeverb. Aussi on constate que le feedback ne peut pas descendre en dessous de 0.23, Cockos considère que $0.23 \approx 0$ et qu'il ne se passe rien d'audible pour des feedbacks plus petits que 0.23.

Dans la même veine un feedback de 1 mènerait à une saturation de la rétroaction et correspondrait à une salle dont les murs sont parfaitement réfléchissant. Cockos limite f_d à $f_d \leq 0.98$

- $f_d \in [0.23; 0.98]$

Sur la figure 21 on remarque comme tout à l'heure une atténuation des hautes fréquences car $d \neq 0$ et on note surtout que le paramètre f_b influence la profondeur du peigne, un feedback grand signifie une augmentation du facteur de qualité de chaque résonnance et agrandi l'écart d'amplitude entre les pics et les creux. Une grande valeur de feedback signifie qu'on laisse plus de signal au fur et à mesure des rétroactions successives par rapport à un feedback plus petit, il en résulte un decay plus long, ainsi la room size influence le temps de réverbération Tr_{60} . Dans la mesure où les combfilters ne sont censés approximer que la partie early reflections de la réverb, on peut légitimement se questionner sur la pertinence de modifier le feedback des combfilters pour rallonger la queue de reverb est à proprement parler du champ diffus. Le champ diffus, ce travail de "floutage" est mené par les filtres passe-tout, c'est donc sur ces derniers qu'on devrait changer les paramètres afin de les rendre plus denses et donc d'augmenter le decay. Serions-nous face à une éventuelle amélioration du code Freeverb ?

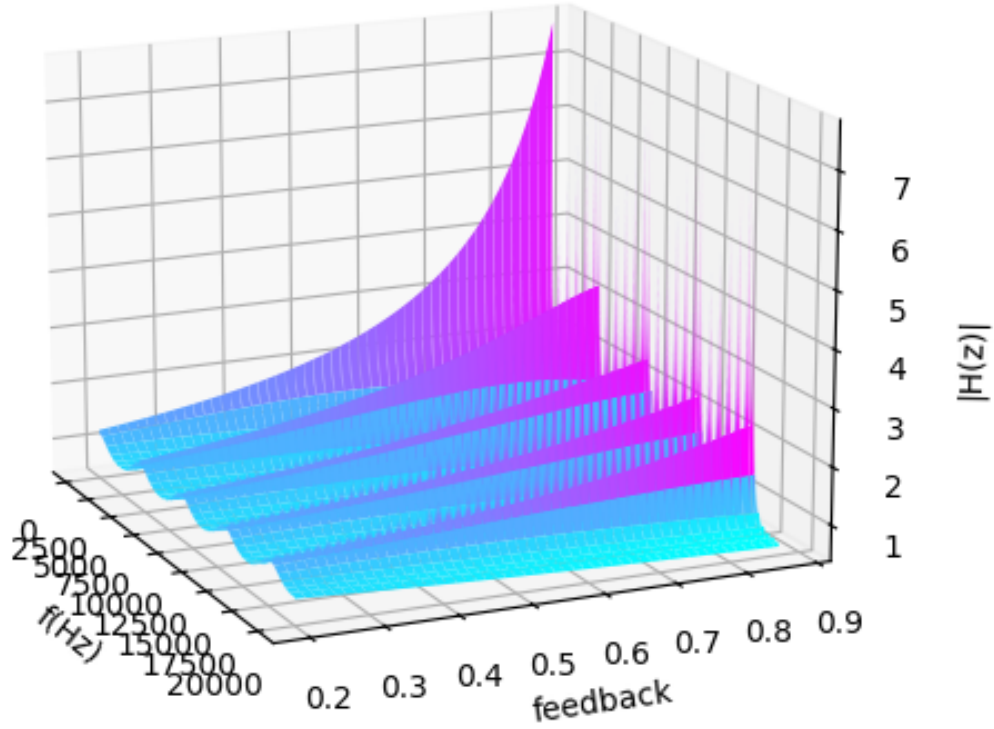


FIGURE 21 – $|H_{cf}(z)|$ en fonction de la fréquence et dépendant du paramètre f_b
 Ce graphique est réalisé pour $d = 0.2$ et $N = 10$, la fréquence f est reliée à la variable z par la relation
 $z = e^{2\pi j f}$

5.4 Conclusion

Au fil de cette analyse nous sommes revenus sur les travaux des scientifiques qui ont posé les bases (et bien plus) des réverbérations algorithmiques, nous avons mis en pratique ces équations à travers une étude de cas particulière au sein de laquelle nous avons décortiqué le code source pour en tirer des conclusions sur la manière dont était construite cette réverb. Cependant bien que la Freeverb soit très bien construite et utilise les résultats découverts au siècle dernier, nous pouvons la remettre en question sous certains aspect. Le tuning des paramètres qui ne semble pas avoir toujours été choisi de manière optimale ainsi que la gestion du decay time en sont des exemples. Ce mythique code a désormais plus de 20 ans et nécessiterais peut-être un coup de jeune. Quoi qu'il en soit, nous espérons que le fonctionnement de cet algorithme est un peu plus clair maintenant et qu'il permettra d'utiliser au mieux les outils de production sur lequel ils reposent.

A Annexe

On peut exprimer **wet1** en fonction de **wet2** en partant du système 19. Pour des soucis de clarté on notera $wet \triangleq W$, $wet1 \triangleq W_1$, $wet2 \triangleq W_2$ et $width = L$

$$\begin{cases} W_1 = W \left(\frac{1+L}{2} \right) \\ W_2 = W \left(\frac{1-L}{2} \right) \end{cases} \iff \begin{cases} W_1 = \frac{W}{2} + \frac{WL}{2} \\ W_2 = \frac{W}{2} - \frac{WL}{2} \end{cases} \quad (27)$$

On additionne et soustrait les deux lignes afin d'obtenir le système :

$$\begin{cases} W_1 - W_2 = WL \\ W_1 + W_2 = W \end{cases} \quad (28)$$

Il vient

$$\begin{cases} \frac{W_1 - W_2}{W} = L \\ W_1 + W_2 = W \end{cases} \iff \begin{cases} \frac{W_1 - W_2}{W_1 + W_2} = L \\ W_1 + W_2 = W \end{cases} \quad \forall W \neq 0 \iff W_1 \neq -W_2 \quad (29)$$

A.1 le code "tuning.h"

```
const int combtuningL1 = 1116;  
const int combtuningR1 = 1116+stereospread;  
const int combtuningL2 = 1188;  
const int combtuningR2 = 1188+stereospread;  
const int combtuningL3 = 1277;  
const int combtuningR3 = 1277+stereospread;  
const int combtuningL4 = 1356;  
const int combtuningR4 = 1356+stereospread;  
const int combtuningL5 = 1422;  
const int combtuningR5 = 1422+stereospread;  
const int combtuningL6 = 1491;  
const int combtuningR6 = 1491+stereospread;  
const int combtuningL7 = 1557;  
const int combtuningR7 = 1557+stereospread;  
const int combtuningL8 = 1617;  
const int combtuningR8 = 1617+stereospread;  
const int allpasstuningL1 = 556;  
const int allpasstuningR1 = 556+stereospread;  
const int allpasstuningL2 = 441;  
const int allpasstuningR2 = 441+stereospread;  
const int allpasstuningL3 = 341;  
const int allpasstuningR3 = 341+stereospread;  
const int allpasstuningL4 = 225;  
const int allpasstuningR4 = 225+stereospread;
```

FIGURE 22 – Longueur des délais en samples pour chaque filtres comb et allpass.
Ces valeurs ont été calculées pour une fréquence d'échantillonnage de 44.1 kHz, elles restent exploitables en 48 kHz mais nécessitent d'être revisitées pour du 88.2 kHz ou du 96 kHz

A.2 La fonction update

Cette fonction 23 met à jour les paramètres internes de la reverb après qu'ils aient été modifiés par l'utilisateur. On peut voir notamment ligne 150 et 151, la mise à jour des **wet1** et **wet2**. A la ligne 166, la boucle for permettant la mise à jour du paramètre de **feedback** des combfilters. A la ligne 172 on trouve la mise à jour du paramètre **damping** pour les combfilters.

```
144 void revmodel::update()
145 {
146     // Recalculate internal values after parameter change
147
148     int i;
149
150     wet1 = wet*(width/2 + 0.5f);
151     wet2 = wet*((1-width)/2);
152
153     if (mode >= freezemode)
154     {
155         roomsize1 = 1;
156         damp1 = 0;
157         gain = muted;
158     }
159     else
160     {
161         roomsize1 = roomsize;
162         damp1 = damp;
163         gain = fixedgain;
164     }
165
166     for(i=0; i<numcombs; i++)
167     {
168         combL[i].setfeedback(roomsize1);
169         combR[i].setfeedback(roomsize1);
170     }
171
172     for(i=0; i<numcombs; i++)
173     {
174         combL[i].setdamp(damp1);
175         combR[i].setdamp(damp1);
176     }
177 }
```

FIGURE 23 – La fonction process.update

A.3 Ressources

Ci-joint les codes Python dont on s'est servi pour faire des plots 2-D et 3-D

A.3.1 Plot 2-D, réponses en phase et amplitude de H_{ap}

```
import matplotlib.pyplot as plt
import numpy as np

N = 6
fb = 0.84
d = 0.2
j = complex(0, 1)

# Définir le gain de  $H_{ap}$  en fonction de la fréquence
def Hgain(f):
    return np.abs(-1 + (fb + 1) * (np.exp(-np.pi*f*j*N))) /
           np.abs(1 - fb * np.exp(-np.pi*f*j))

# Définir l'argument de  $H_{ap}$  en fonction de la fréquence
def Harg(f):
    return np.angle((-1 + (fb + 1) * (np.exp(-np.pi*f*j*N))) /
                    (1 - fb * np.exp(-np.pi*f*j)))

# Définir la liste de valeur f
f = np.linspace(0, 20000, 10000)

# Stocker les valeurs des equations dans des listes
H_gain = Hgain(f)
H_arg = np.mod(Harg(f)*180/np.pi, -360)

# creer le graphique du gain
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1) # Creer une premiere sous-figure
plt.scatter(f, H_gain, label='|H(z)|', color='red',
            alpha=0.6, s=0.1) # Utiliser alpha pour la transparence
plt.xlabel('Frequence (Hz)')
plt.ylabel('|H(z)|')
plt.title('Reponse en amplitude du filtre passe-tout')
plt.legend()

# creer le graphique de l'argument
plt.subplot(1, 2, 2) # Creer une deuxieme sous-figure
plt.scatter(f, H_arg, label='arg(H(z))', color='blue',
            alpha=0.6, s=0.1) # Utiliser alpha pour la transparence
plt.xlabel('Frequence (Hz)')
plt.ylabel('arg(H(z))')
plt.title('Reponse en phase du filtre passe-tout')
plt.legend()
```

```
# Ajuster la mise en page pour eviter que les graphiques  
se chevauchent  
plt.tight_layout()  
  
# Afficher le graphique  
plt.show()
```

A.3.2 Plot 3-D représentation de l'amplitude de $H_{cf}(z)$ en fonction de la fréquence et dépendant du paramètre f_b ou d

```

import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

N = 2
fb = 0.84
## b=0.2
j = complex(0, 1)

# Définir l'équation en fonction de deux variables (f, d)
def equation(f, d):
    return np.abs(np.exp(-np.pi*f*j*N))/(np.abs(np.exp(np.pi*f*j*N)
    - np.exp(-np.pi*f*j*N)*((fb*(1-d))/(1-d*np.exp(-np.pi*f*j))))))

## def equation_fb(f, fb):
    return np.abs(np.exp(-np.pi*f*j*N))/(np.abs(np.exp(np.pi*f*j*N)
    - np.exp(-np.pi*f*j*N)*((fb*(1-d))/(1-d*np.exp(-np.pi*f*j))))))

# Créer une grille de valeurs pour f d et fb
f = np.linspace(0, 20000, 100)
d = np.linspace(0, 1, 100)
## fb = np.linspace(0.1, 0.98, 100)

# Créer une grille 2D pour f et d et f et fb
f, d = np.meshgrid(f, d)
## f, b = np.meshgrid(f, fb)

# Calculer les valeurs de l'équation pour chaque paire (f, d)
(ou (f, fb)
z = equation(f, d)
## z = equation(f, fb)

# Créer une figure 3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Tracer la surface 3D
ax.plot_surface(f, d, z, cmap='viridis')
## ax.plot_surface(f, fb, z, cmap='viridis')

# Ajouter des étiquettes
ax.set_xlabel('f(Hz)')
ax.set_ylabel('d')
ax.set_zlabel('|H(z)|')
# Afficher le graphique
plt.show()

```

Références

- [1] M. R. Schroede, “Natural Sounding Artificial Reverberation,” 1962. (Cité en page 3.)
- [2] J. A. Moorer, “About This Reverberation Business,” *Computer Music Journal*, vol. 3, no. 2, pp. 13–28, 1979. Publisher : The MIT Press. (Cité en page 6.)
- [3] J. O.SMITH III, “Physical audio signal processing for virtual musical instruments and audio effects,” 2010. (Cité en page 7.)
- [4] M. R. Schroeder, “Digital Simulation of Sound Transmission in Reverberant Spaces,” *The Journal of the Acoustical Society of America*, vol. 45, pp. 424–431, Aug. 1969. (Cité en page 8.)
- [5] M. Schroeder and B. Logan, “”Colorless” artificial reverberation,” *IRE Transactions on Audio*, vol. AU-9, pp. 209–214, Nov. 1961. Conference Name : IRE Transactions on Audio. (Non cité.)
- [6] WolfSound, “Allpass Filter : All You Need To Know.” (Non cité.)
- [7] J.-M. Jot, “Efficient models for reverberation and distance rendering in computer music and virtual audio reality © Ircam, 1996-2008, Paris, France.” (Non cité.)